SignSpeak - System Architecture

System overview

SignSpeak is a web-based AI application designed to translate American Sign Language (ASL) into natural English text and spoken voice in real-time. The system operates as a **universal overlay** compatible with any video conferencing platform (Google Meet, Zoom, Microsoft Teams). Designed for privacy and low latency, the browser captures webcam video and runs **MediaPipe** locally to extract hand landmarks. Instead of streaming heavy raw video, only lightweight coordinate data is transmitted to the backend. The backend coordinates machine learning inference and language reconstruction, broadcasting the final translated sentence (text and audio triggers) to all participants in the session via **WebSocket**.

Primary goals:

- **Privacy-First Architecture:** No raw video ever leaves the client's browser. Only abstract hand coordinates (landmarks) are transmitted, ensuring user privacy and significantly reducing bandwidth usage.
- **Universal Compatibility:** Instead of relying on platform-specific plugins (like Google Meet Add-ons), the system uses **Document Picture-in-Picture (PiP)** to create a floating translation window that stays on top of any application.
- **Dual-Role Accessibility:** Supports two distinct modes:
    - **Signer Mode:** Captures input and broadcasts translation.
    - **Listener Mode:** Receives text and **Text-to-Speech (TTS)** audio without needing a webcam.
- **Contextual Translation:** Converts raw gesture sequences into fluent English sentences using an LLM.

Architecture Summary

The system consists of three main containers:

1. Frontend Web App (React + Vite) The user interface running in the browser. It is responsible for:

- **Role Management:** Handling "Signer" (Webcam ON) vs. "Listener" (Webcam OFF) logic.
- **Local Processing:** Running the **MediaPipe** framework via WebAssembly to extract hand landmarks client-side.
- **PiP Management:** Detaching the UI into an "Always-on-top" floating window for seamless integration with video calls.
- **Output Rendering:** Displaying the translated text and synthesizing speech (TTS) via the browser API.

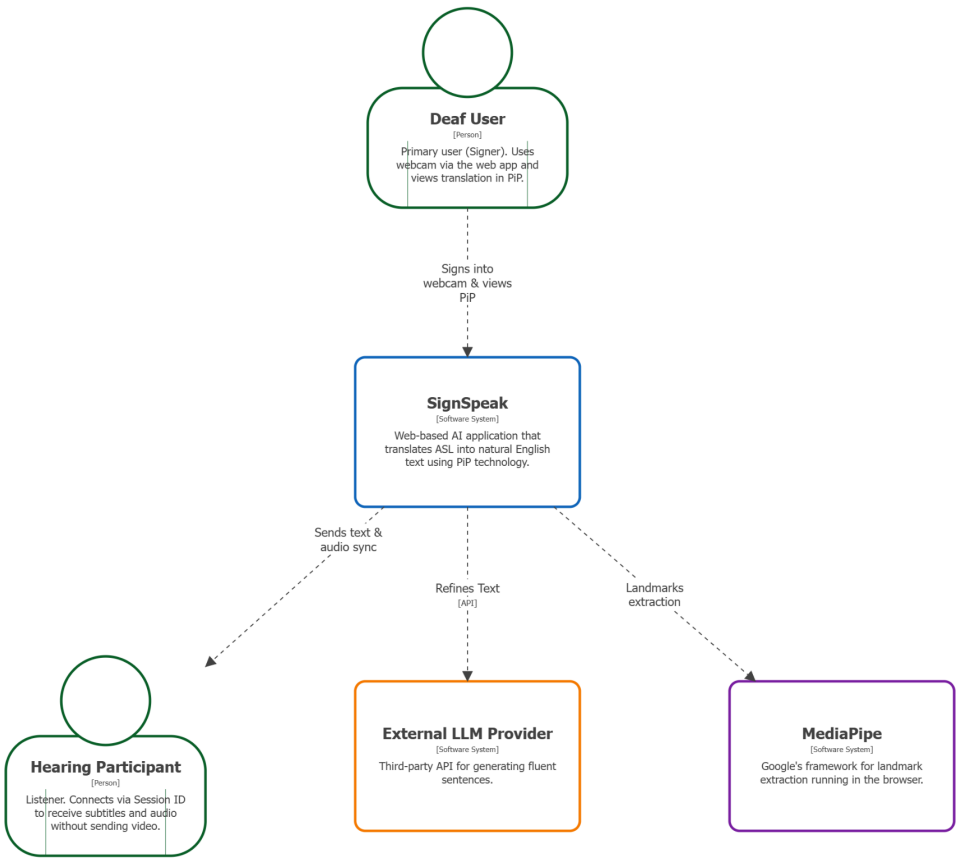
2. Backend API (Java Spring Boot) The central orchestrator of the system. It is responsible for:

- **Session Synchronization:** Managing **WebSocket (STOMP)** connections to ensure all participants in a meeting (Signers and Listeners) receive updates instantly.
- **Data Buffering:** Accumulating the stream of incoming landmarks into time-windowed buffers.
- **Delegation:** Forwarding buffered data to the ML Service and relaying the final response back to the frontend clients.

3. Machine Learning Service (Python/PyTorch) The intelligence engine that translates movement into language in a two-step process:

1. **Sign Recognition (Classification):** A specialized deep learning model analyzes the sequence of hand landmarks to identify individual ASL signs (glosses).
2. **Sentence Reconstruction (LLM):** A Large Language Model (LLM) takes this raw sequence of glosses (e.g., "ME GO STORE") and rewrites it into a grammatically correct English sentence (e.g., "I am going to the store"). This separation of concerns separates the task of *visual perception* from the task of *language understanding*, maximizing overall accuracy.

Level 1: Context (Actors & External Systems)



SignSpeak - Level 1: System Context
domenica 7 dicembre 2025 alle ore 18:36 Ora standard dell'Europa centrale

The Level 1 Context diagram illustrates the high-level vision of the SignSpeak system. It treats the entire system as a single "black box" (the blue `SignSpeak` box) and focuses exclusively on its interactions with its users (Actors) and other software systems (External Systems).

The purpose of this diagram is to define the system's boundaries and scope, showing who uses it and which external dependencies it relies on to function.

Diagram Elements

Actors: Actors represent the human users who interact with the system.

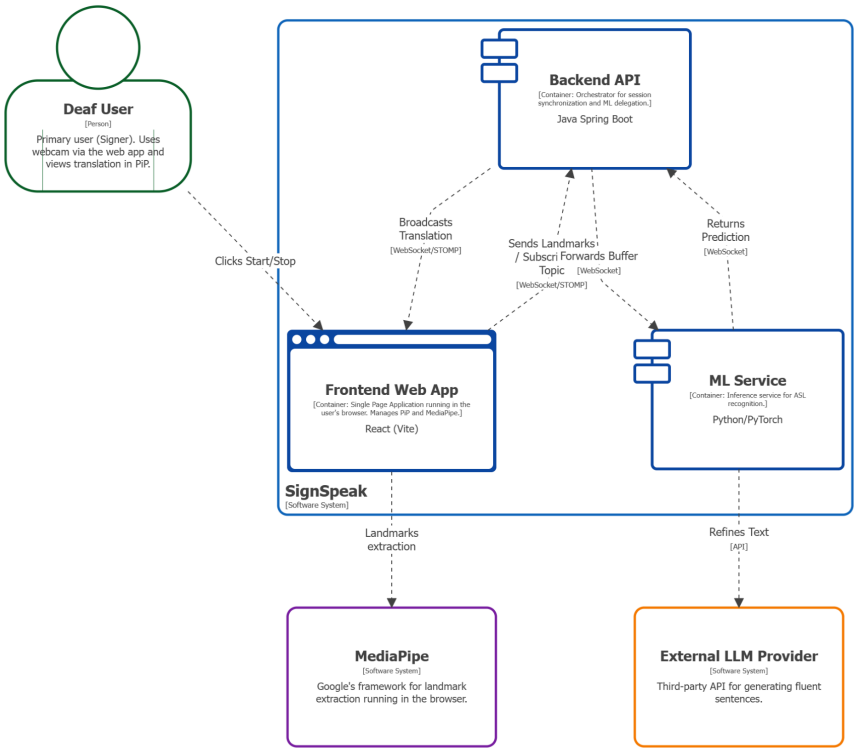
- **Deaf User :** This is the primary user.
    - **Role:** Starts the session, activates the webcam, and signs.
    - **Interaction:** The relationship is bidirectional ("Signs into webcam & views PiP"). The user provides video input for analysis and simultaneously views the translated output in a detached Picture-in-Picture window overlay.
- **Hearing Participant (Listener):** This is the secondary user.
    - **Role:** Joins the session using a unique Session ID to understand the signer.
    - **Interaction:** This user is **connected directly** to the system ("Sends text & audio sync"). Their device receives real-time data from SignSpeak to display subtitles and trigger Text-to-Speech audio locally, without sending their own video.

Internal System SignSpeak : This is the system we are building. The description clarifies its primary purpose: to translate ASL into English text.

External Systems: These are the third-party software systems our system depends on. They have been color-coded using tags for better categorization.

- **External LLM Provider (Orange):** Refines Text. The system sends raw glosses (basic sign words) to this API, which returns grammatically correct, fluent English sentences.
- **MediaPipeFramework (Purple):** Landmarks extraction**.**The external framework actively used by the frontend to perform real-time hand landmark extraction. It is modeled as an *External System* (rather than an internal component) because, although it runs client-side, it is a complex, pre-trained framework that the SignSpeak team does not own or maintain. This modeling decision highlights the system's critical dependency on this third-party technology for its core functionality.

Level 2: Containers



SignSpeak - Level 2: Containers
domenica 7 dicembre 2025 alle ore 18:36 Ora standard dell'Europa centrale

This diagram breaks down the SignSpeak system into its main executable services and shows how they communicate.
The Level 2 Container diagram "zooms in" on the `SignSpeak` system box defined in Level 1. It reveals the main, deployable, and runnable pieces of software (the "containers") that make up the system: a frontend web app, a backend API, and a machine learning service. The solid blue boundary line (`SignSpeak System`) indicates that these containers are all part of the single system we are building.

Containers and Data Flow

Internal Containers (Our System)

- **Frontend Web App (React + Vite):** This is the user's entry point, running in their browser.It manages the **Picture-in-Picture (PiP)** window and the webcam feed. Crucially, it loads the external **MediaPipe** framework locally to perform landmark extraction on the client side. It sends the extracted landmarks to the Backend and simultaneously **subscribes to a specific session topic** via **WebSocket (STOMP)** to receive translated text and audio triggers broadcasted by the system.
- **Backend API (Java Spring Boot):** This is the central orchestrator of the system. Its primary role is **Session Sync**. It receives landmark streams from Signers, buffers the data, and forwards it to the ML Service. It forwards raw buffers to the ML Service and

broadcasts the final returned predictions to all connected clients (Signers and Listeners) subscribed to the session topic.

- **ML Service (Python/PyTorch):** This is the AI "brain" of the system. It receives the buffered landmark stream from the Backend via `WebSocket` and runs the gesture recognition pipeline (PyTorch) to convert landmarks into ASL glosses.
- **Output Strategy (Dual-Feedback):**
    1. **Partial Updates:** As soon as individual signs are recognized (e.g., "ME", "GO"), it streams them word-by-word to the Backend. This provides immediate, low-latency feedback to the user.
    2. **Final Refinement:** Once a sequence is completed, it calls the **External LLM Provider** via a synchronous API to refine the rough glosses into a fluent English sentence. This final result is then sent to the Backend to replace the partial words.

Technology and Data Flow

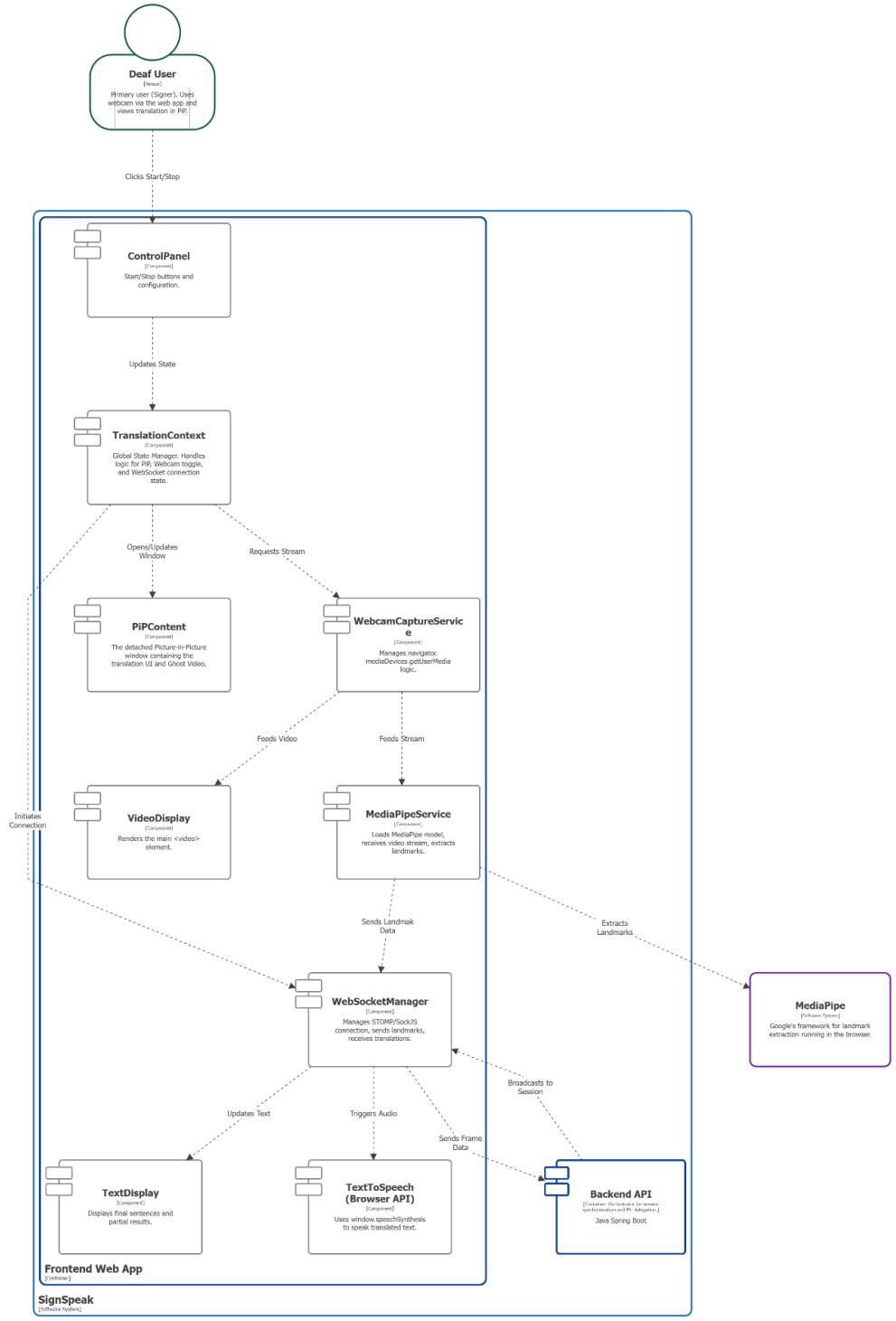
The diagram clearly defines the technologies used for communication, which is crucial for our low-latency goal:

- **WebSocket (Real-time):** The core data path (`Frontend -> Backend -> ML Service` and back) uses `WebSocket`. This is a deliberate choice to maintain a persistent, low-latency connection required for real-time streaming.
- **API (Synchronous):** Used specifically for the **ML Service** -> **External LLM** connection ("Refines Text"), where the system waits for the LLM to generate a fluent sentence.
- **Local Execution:** The link between Frontend and **MediaPipe** represents a local function call, ensuring that raw video processing happens instantly on the user's device without network latency.

Level 3: Components

The Level 3 diagrams zoom into each container from Level 2 to show the main logical components (modules or services) *inside* them and how they interact.

Frontend Web App Components

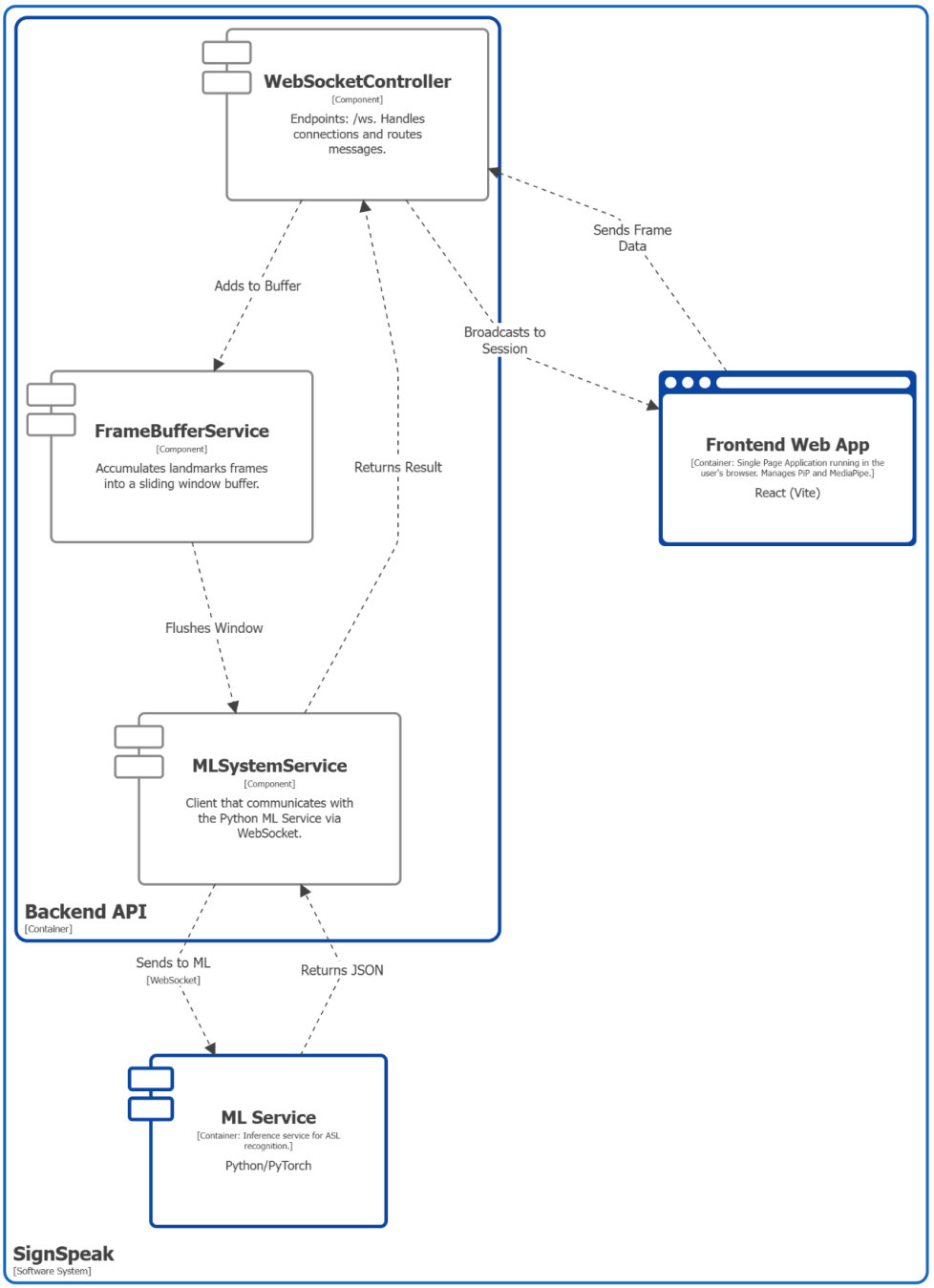


SignSpeak - Level 3: Frontend Components
domenica 7 dicembre 2025 alle ore 18:36 Ora standard dell'Europa centrale

This diagram shows the internal structure of the `Frontend Web App`. The flow is initiated by the user and follows three main paths: user control and data processing.

- **User Control Flow:** The `Deaf User` interacts with the `ControlPanel` (e.g., "Start/Stop" buttons). This action updates the **TranslationContext**. This component acts as the global state manager and the "brain" of the frontend.
  Once active, the **TranslationContext** orchestrates three parallel actions:
  1. It triggers the **WebcamCaptureService** to access the camera.
  2. It opens or updates the **PiPContent**, which creates the detached floating window for the UI.
  3. It initiates the connection via the **WebSocketManager**.
- **Data Processing Flow:**
  1. The `WebcamCaptureService` (once triggered) provides the video stream to both the `VideoDisplay` component (for the user to see themselves) and the `MediaPipeService`.
  2. The `MediaPipeService` is a wrapper that calls the external `MediaPipe` framework to perform landmark extraction on the video stream.
  3. It sends the resulting "Landmark Packets" to the `WebSocketManager`.
  4. The `WebSocketManager` sends this data to the `Backend API` over the secure WebSocket connection.
- **Output Flow**:When the `Backend API` broadcasts a result (partial words or final sentences), the`WebSocketManager` receives the message.It immediately updates the TextDisplay component to render the translation. For **TextToSpeech (Browser API)**, the user can manually click the **"Speaker" button** in the interface (ControlPanel/PiPContent) to trigger the audio playback of the translation.

Backend API Components

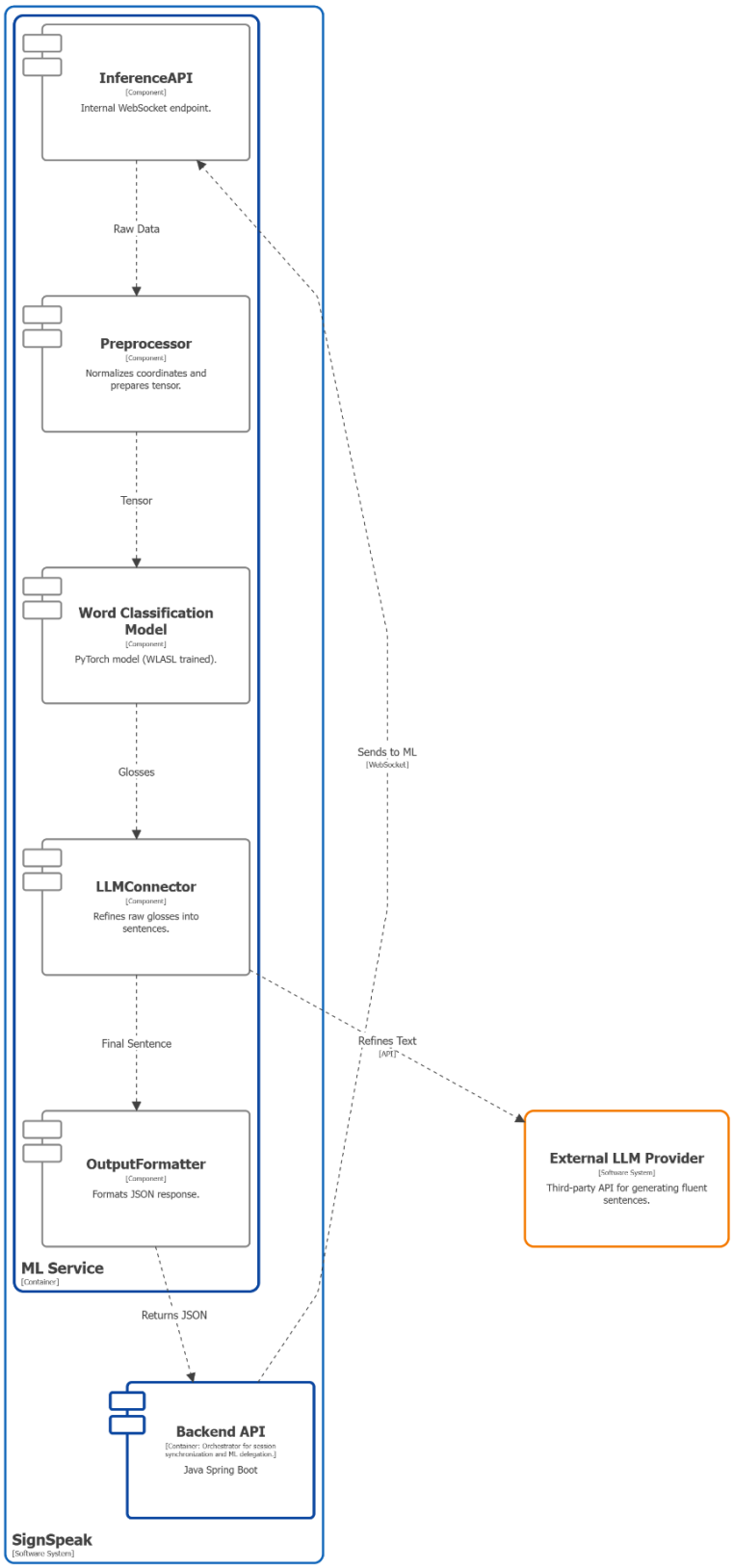


SignSpeak - Level 3: Backend Components
domenica 7 dicembre 2025 alle ore 18:36 Ora standard dell'Europa centrale

This diagram details the `Backend API`, which acts as the system's central orchestrator for data synchronization and ML delegation.

- **Data Ingest:** The `WebSocketController` is the entry point. It manages the STOMP endpoints (`/ws`) and handles incoming connections. It receives raw landmarks from the `FrontendWebApp` and immediately routes it to the buffering service.
- **Buffering:** The `FrameBufferService` is responsible for temporal consistency. It accumulates the incoming landmark frames into a sliding window buffer. It determines when a sequence is "complete" or ready for processing and "Flushes the Window" to the next stage.
- **ML:** The `MLSystemService` acts as the internal client for the AI engine. It maintains a dedicated WebSocket connection to the external `MLService` (Python). It forwards the flushed buffers to the ML Service and waits for the inference result.
- **Response & Broadcasting:** When the `MLService` returns a prediction (Partial word or Final JSON), the `MLSystemService` receives it and passes the result back to the `WebSocketController`. It then broadcasts this message to the specific session topic, ensuring that both the Signer and any Listeners receive the update simultaneously.

ML Service Components

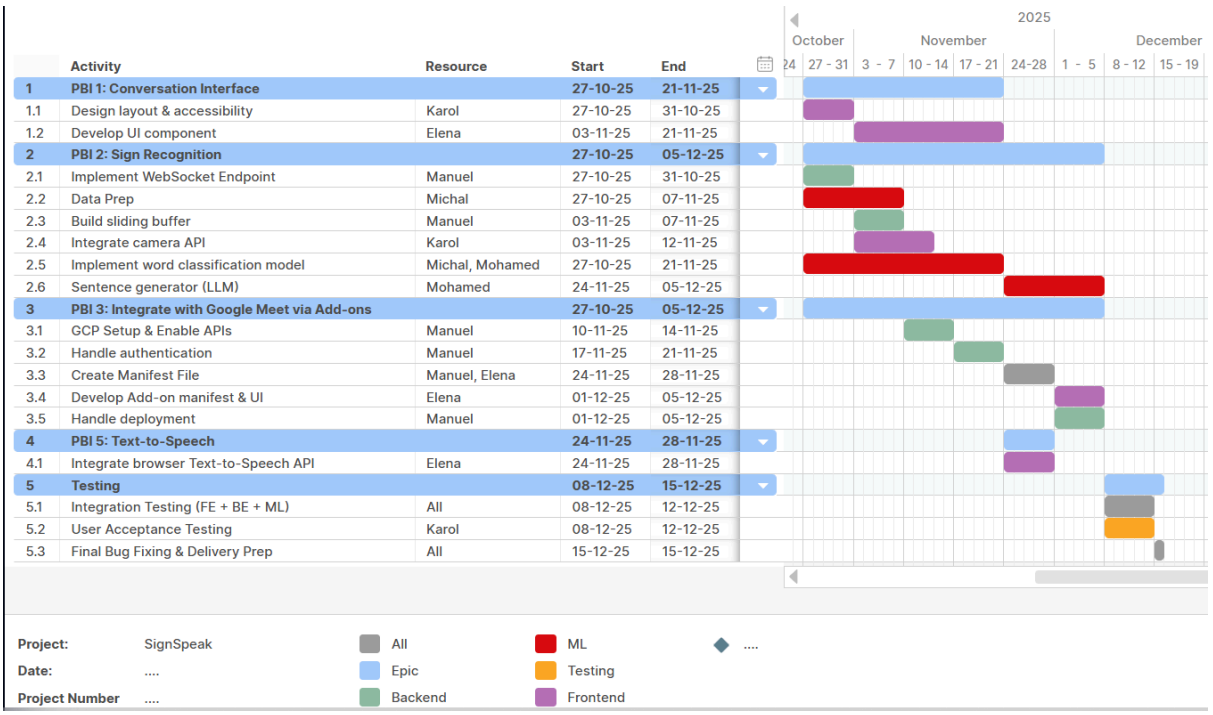


SignSpeak - Level 3: ML Service Components
domenica 7 dicembre 2025 alle ore 18:36 Ora standard dell'Europa centrale

This diagram illustrates the internal AI pipeline within the `ML Service` container.

1. **Entrypoint:** The `InferenceAPI` (a WebSocket endpoint) receives the landmark stream from the `Backend API`.
2. **Preprocessing:** It sends the "Raw landmarks" to the `Preprocessor`, which cleans the data and converts it into the "Input Tensor" (T x F) required by the model.
3. **Recognition:** The tensor is passed to the `Word classification Model` (our GRU+CTC model), which outputs "Token Probabilities".
4. **Language Formation:** These probabilities (or tokens) are sent directly to the `LLMConnector`. The `LLMConnector` is responsible for calling the `External LLM Provider` with an engineered prompt to transform the ASL tokens into a "Fluent Sentence".
5. **Formatting:** The fluent sentence is passed to the `OutputFormatter`, which packages the final "JSON Response".
6. **Return:** The `OutputFormatter` sends the response back to the `InferenceAPI`, which, in turn, sends the "final translated sentence" back to the `Backend API` over the WebSocket connection, completing the flow.

Gantt Dependencies & Schedule Alignment



	Activity	Resource	Start	End
1	**PBI 1: Conversation Interface**		27-10-25	21-11-25
1.1	Design layout & accessibility	Karol	27-10-25	31-10-25
1.2	Develop UI component	Elena	03-11-25	21-11-25
2	**PBI 2: Sign Recognition**		27-10-25	05-12-25
2.1	Implement WebSocket Endpoint	Manuel	27-10-25	31-10-25
2.2	Data Prep	Michal	27-10-25	07-11-25
2.3	Build sliding buffer	Manuel	03-11-25	07-11-25
2.4	Integrate camera API	Karol	03-11-25	12-11-25
2.5	Implement word classification model	Michal, Mohamed	27-10-25	21-11-25
2.6	Sentence generator (LLM)	Mohamed	24-11-25	05-12-25
3	**PBI 3: Integrate with Google Meet via Add-ons**		27-10-25	05-12-25
3.1	GCP Setup & Enable APIs	Manuel	10-11-25	14-11-25
3.2	Handle authentication	Manuel	17-11-25	21-11-25
3.3	Create Manifest File	Manuel, Elena	24-11-25	28-11-25
3.4	Develop Add-on manifest & UI	Elena	01-12-25	05-12-25
3.5	Handle deployment	Manuel	01-12-25	05-12-25
4	**PBI 5: Text-to-Speech**		24-11-25	28-11-25
4.1	Integrate browser Text-to-Speech API	Elena	24-11-25	28-11-25
5	**Testing**		08-12-25	15-12-25
5.1	Integration Testing (FE + BE + ML)	All	08-12-25	12-12-25
5.2	User Acceptance Testing	Karol	08-12-25	12-12-25
5.3	Final Bug Fixing & Delivery Prep	All	15-12-25	15-12-25